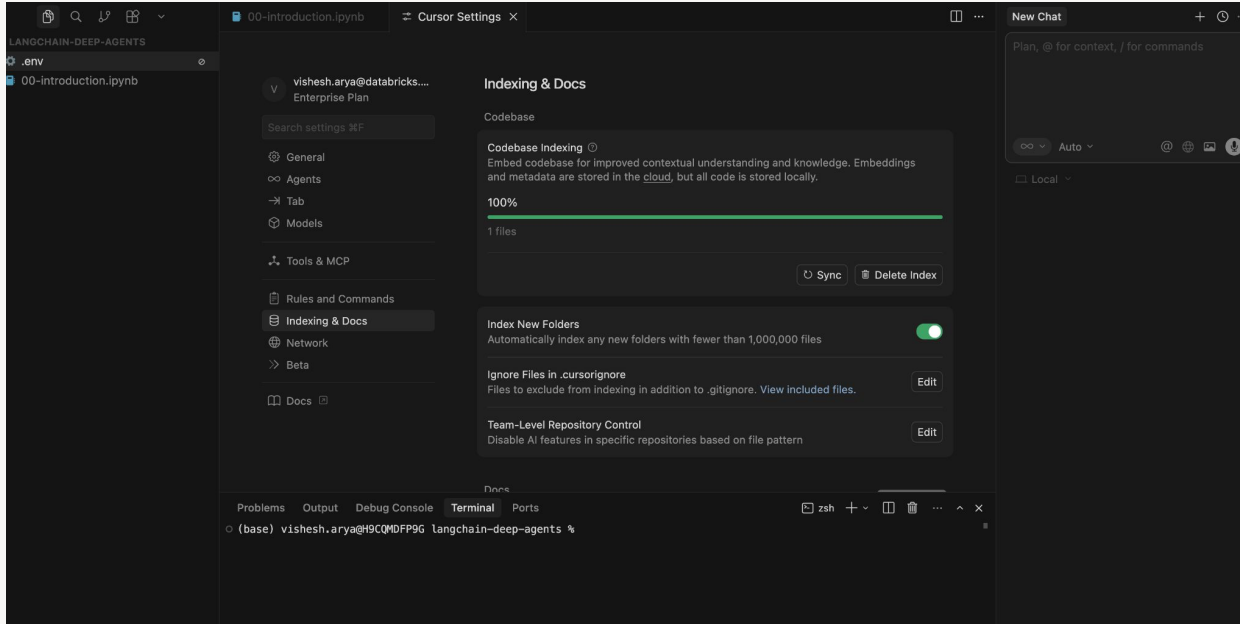


Vibe Coding with Cursor AI

Introduction to Cursor AI

Cursor Interface

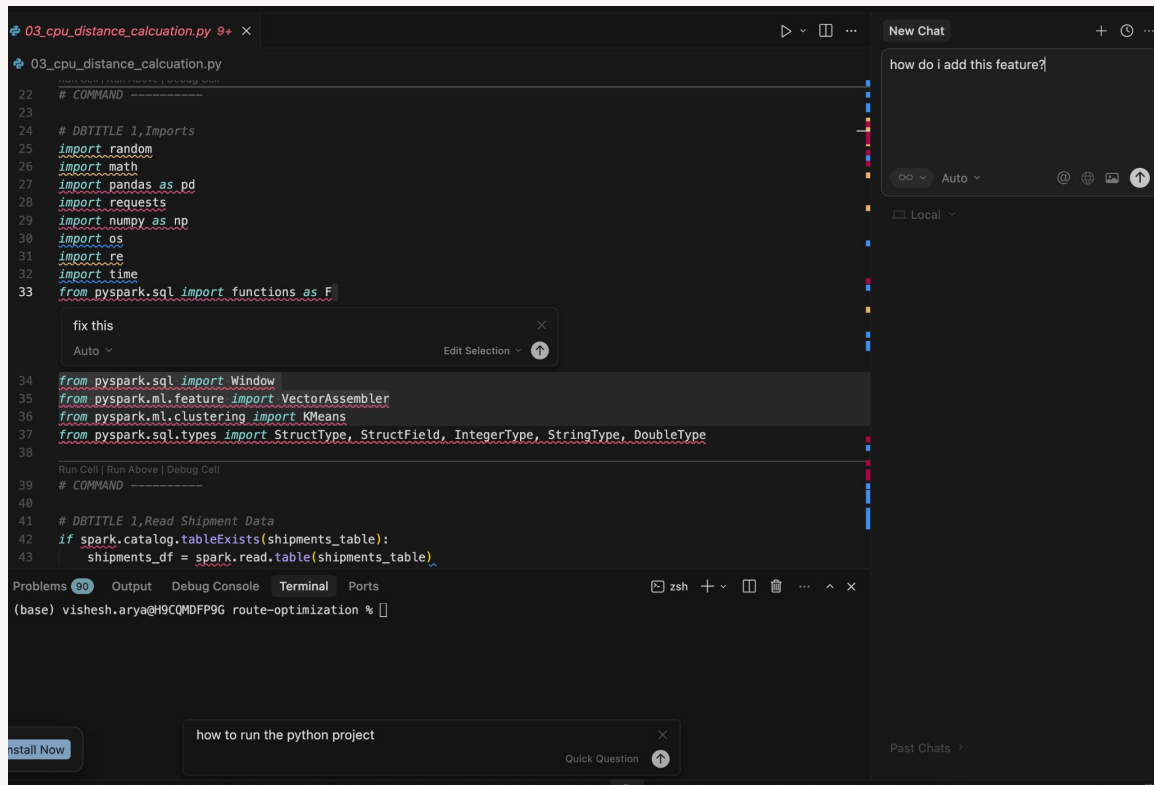
Settings Tab



- Let's you configure AI interaction settings
- **Codebase Indexing** helps AI generate better responses by indexing codebase
- With codebase indexing, Cursor automatically keeps the indexes updated when new files are added
- You can add custom **docs** as context for your projects
- Choose different AI models to power code agent
- **Rules** define how AI interacts with code
- Use **MCP** to connect cursor to external systems and data sources

Cursor Interface

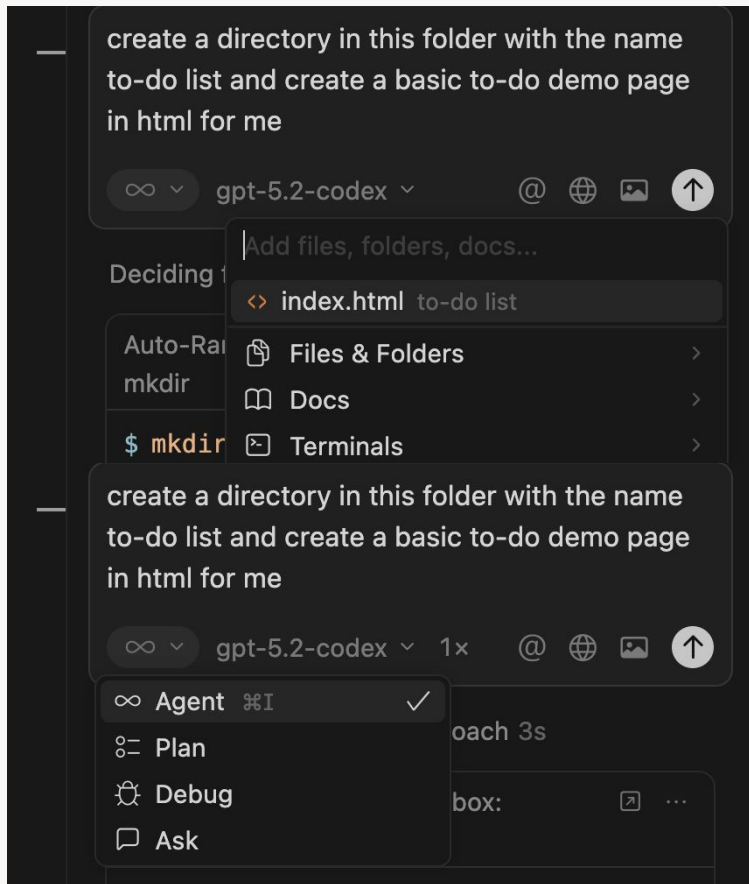
AI Assistance – Inline Editing, Command Assistance, AI Chat Panel



- Use **Inline code editing** by highlighting the code and using Cmd + K (Mac) / Ctrl + K (Windows)
- You can also use terminal to run commands directly or Cmd + K (Mac) / Ctrl + K (Windows) to use **AI command assistance** to generate the right command
- Cursor understands entire codebase, find the right files, and can edit multiple files intelligently at once via prompts in **Chat Panel**

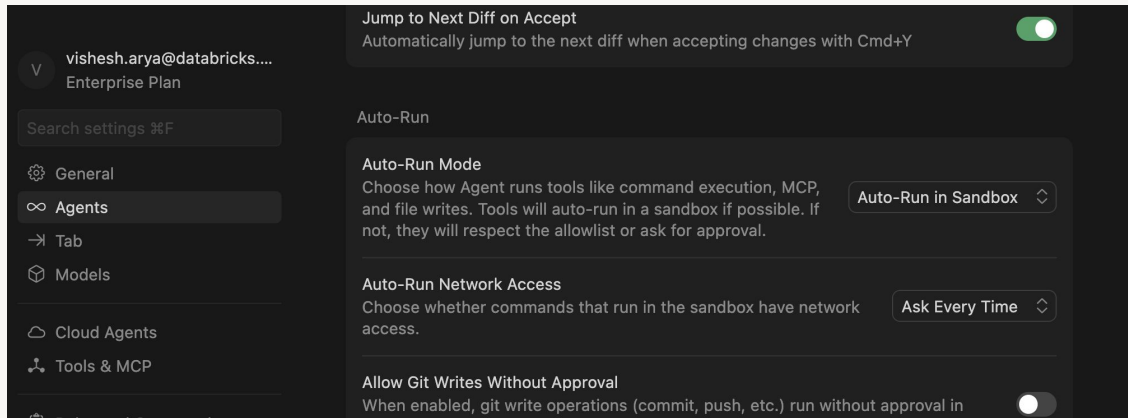
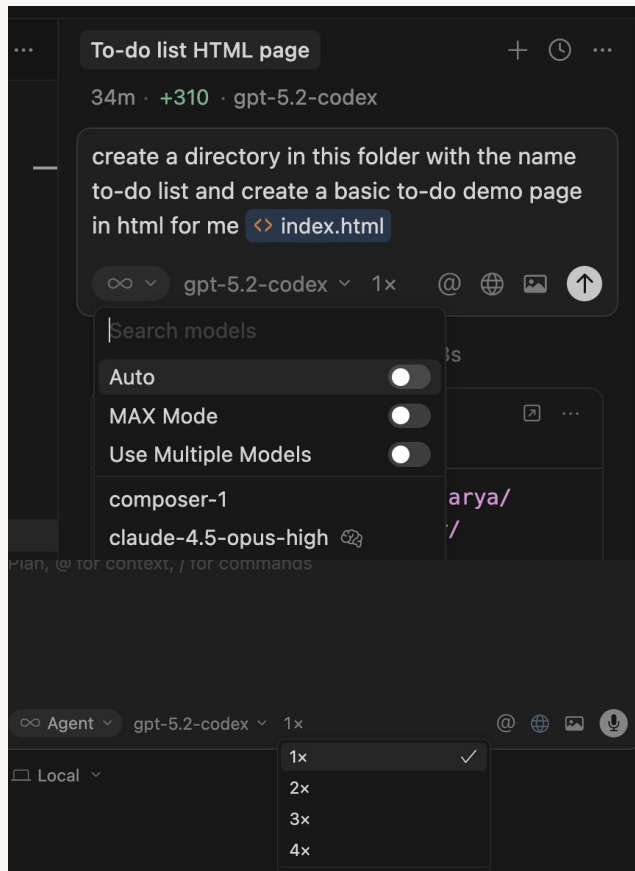
AI Powered Development

AI Chat Panel



- We can add a bunch of additional context in the form of files, docs, etc. by using **@ (add context)**
- You can also provide additional context by enabling web search tool or inserting screenshot / images
- AI Chat Panel Modes:
 - **Agent:** AI agent using tools and reasoning to perform coding tasks with minimal supervision
 - **Ask:** ask questions about the code, get explanations, and discover codebase
 - **Plan:** create detailed plan for accomplishing complex tasks
 - **Debug:** systematically diagnose and fix bugs using runtime traces
 - During a conversation, you can also switch between these modes

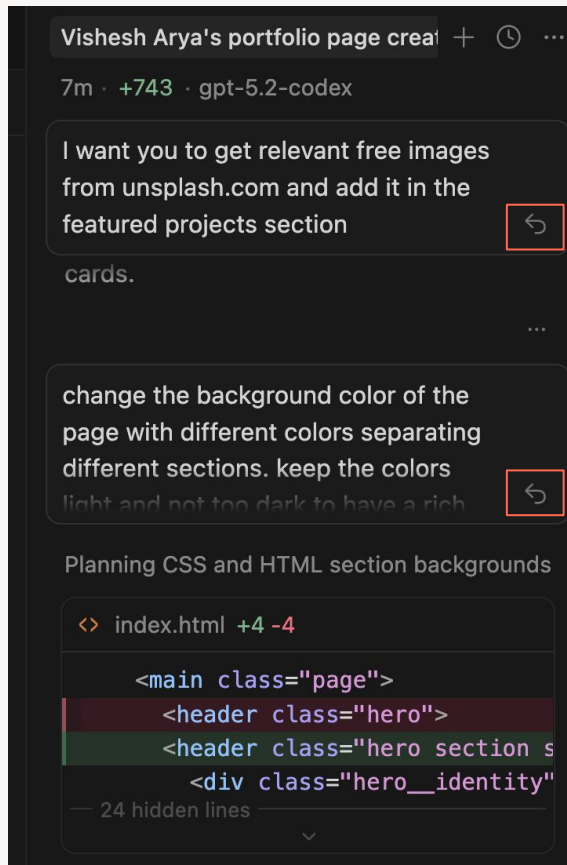
AI Chat Panel



- Select LLM Models to choose from
- Cursor has its own AI models but it also integrates with SOTA models from OpenAI, Anthropic, and Google
- We can add more model options from Cursor Settings > Models tab
- Enable Auto-Run mode to allow the Agent run tools and commands automatically without asking for permission
- **Cycle Agent Count** uses multiple agents on the same task and keep best solution. Each model does consume tokens

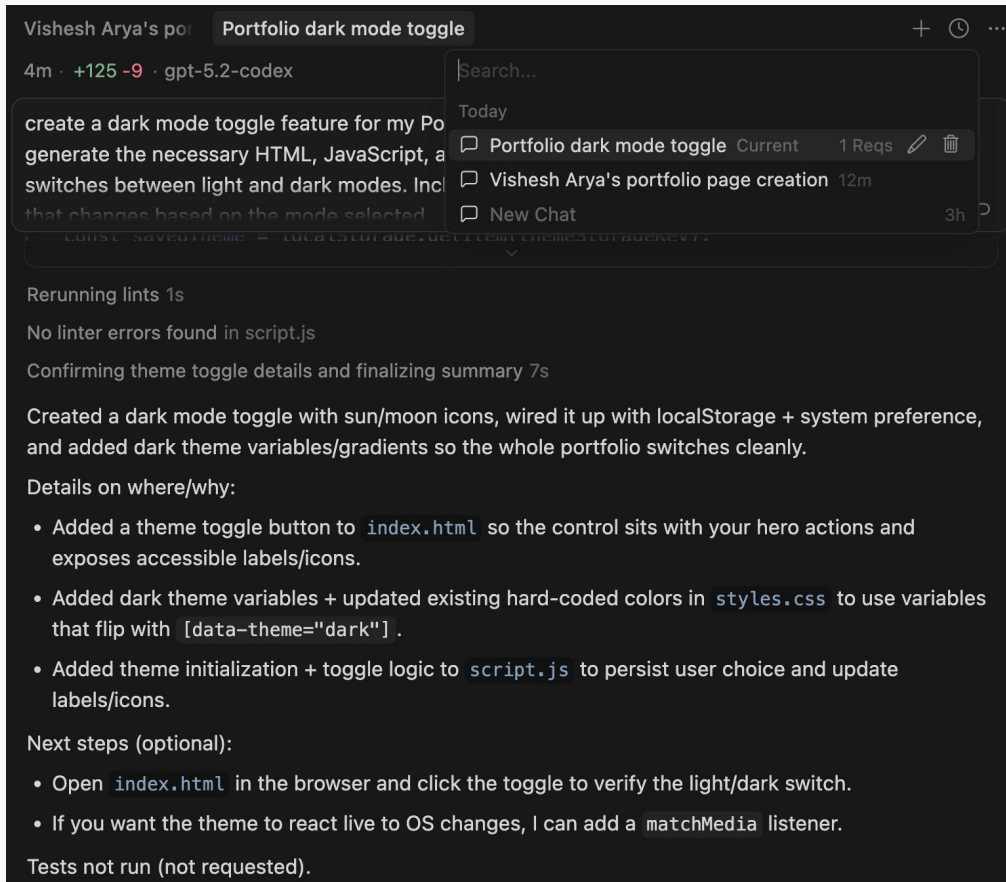
Debugging & Iteration

Debugging & Iteration



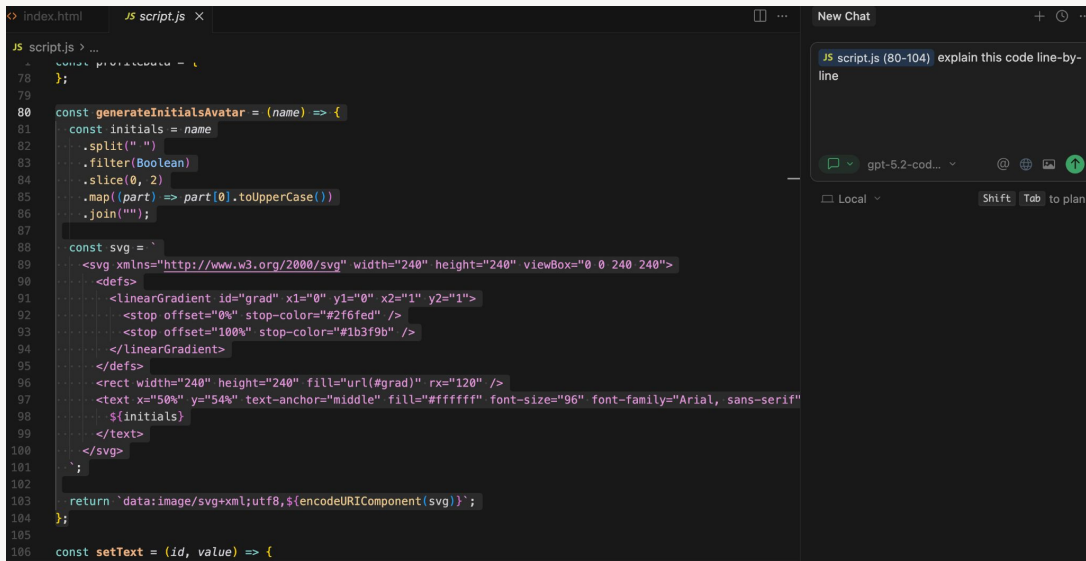
- Go to ↩ (**Restore Checkpoint**) in the chat to revert code changes to a previous saved state
- Use this to remove unwanted edits or return to previously working version
- Use the new **Debug** feature by Cursor

Building New Features



- While sending messages in an ongoing chat, AI remembers the context which is super useful when iterating on a related feature
- However, for building entirely new feature, keeping this old context can lead to errors
- In Cursor, start a fresh chat for each new feature to reset the context
- Cursor also allows to view Chat History using  (**Show Chat History**) to help stay organized on specific features

Ask vs Agent Mode



The screenshot shows a code editor with a dark theme. The main editor displays a JavaScript file named `script.js` with the following code:

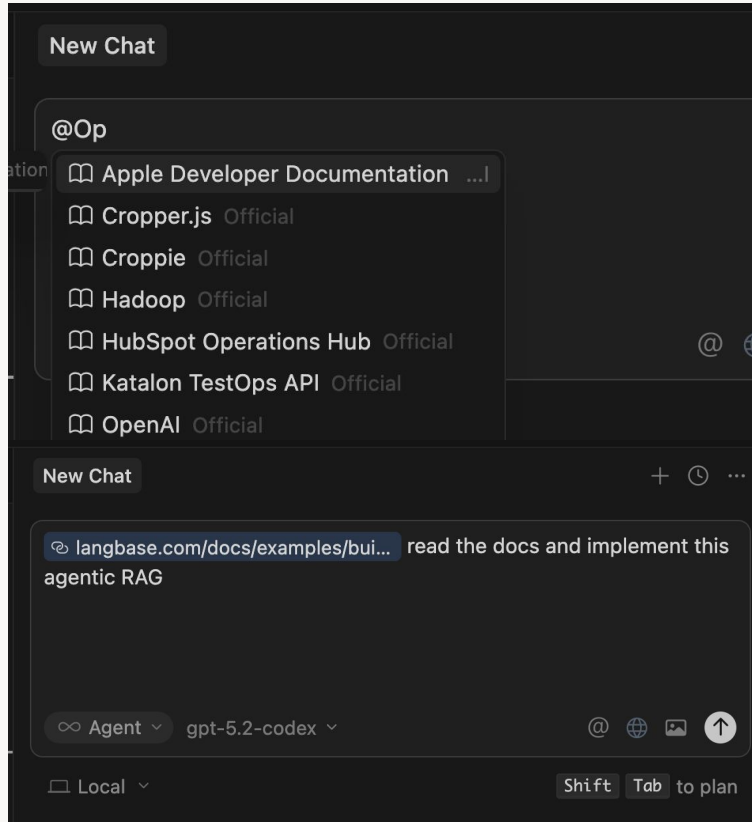
```
100 const generateInitialsAvatar = (name) => {  
101   const initials = name  
102     .split(" ")  
103     .filter(Boolean)  
104     .slice(0, 2)  
105     .map((part) => part[0].toUpperCase())  
106     .join("");  
107  
108   const svg = `  
109     <svg xmlns="http://www.w3.org/2000/svg" width="240" height="240" viewBox="0 0 240 240">  
110       <defs>  
111         <linearGradient id="grad" x1="0%" y1="0%" x2="100%" y2="100%">  
112           <stop offset="0%" stop-color="#2f6f6f" />  
113           <stop offset="100%" stop-color="#1b3f9b" />  
114         </linearGradient>  
115       </defs>  
116       <rect width="240" height="240" fill="url(#grad)" rx="120" />  
117       <text x="50%" y="54%" text-anchor="middle" fill="ffffff" font-size="96" font-family="Arial, sans-serif">  
118         ${initials}</text>  
119     </svg>  
120   `;  
121  
122   return `data:image/svg+xml;utf8,${encodeURIComponent(svg)}`;  
123 }  
124  
125  
126 const setText = (id, value) => {
```

On the right side, there is a "New Chat" window. It contains the text: "JS script.js (80-104) explain this code line-by-line". Below the text, there are icons for a chat bubble, a search icon, a plus icon, and a green up arrow icon. At the bottom of the chat window, there is a dropdown menu showing "Local" and buttons for "Shift" and "Tab to plan".

- Use Cmd + I (Mac) / Ctrl + I (Windows) to add selected code lines to chat context
- Unlike the agent mode that uses tools and reasoning to performing coding tasks, with ask mode you can ask questions about code, get explanations, and discover codebase
- You can also ask follow-up questions for further explanation

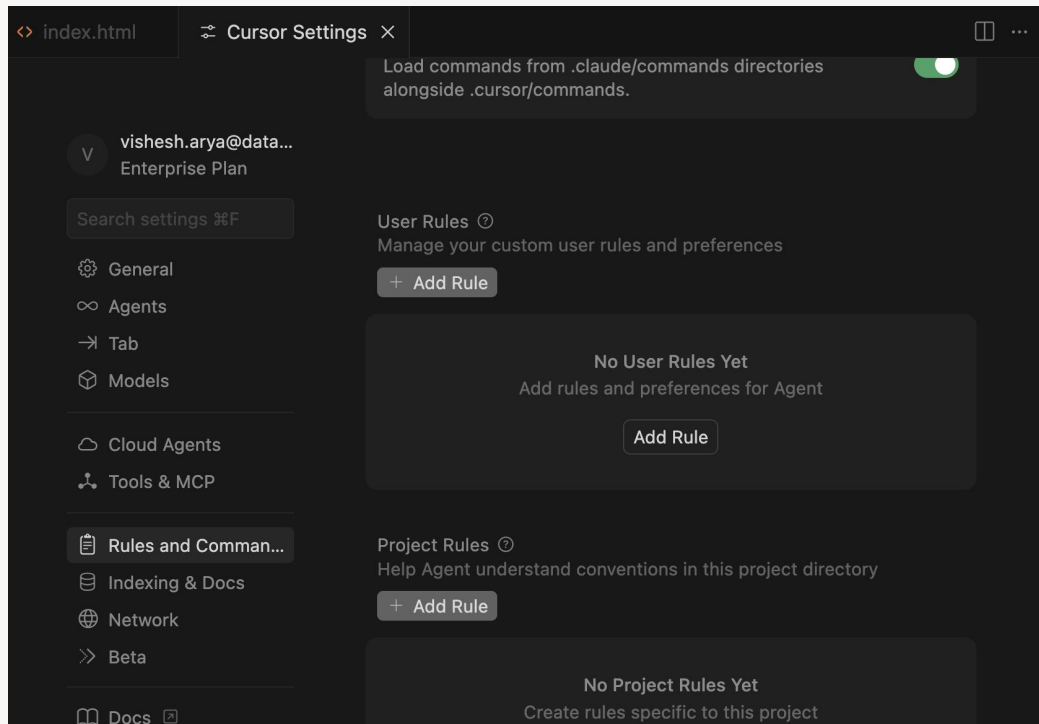
Advanced Cursor Features

Context Tools



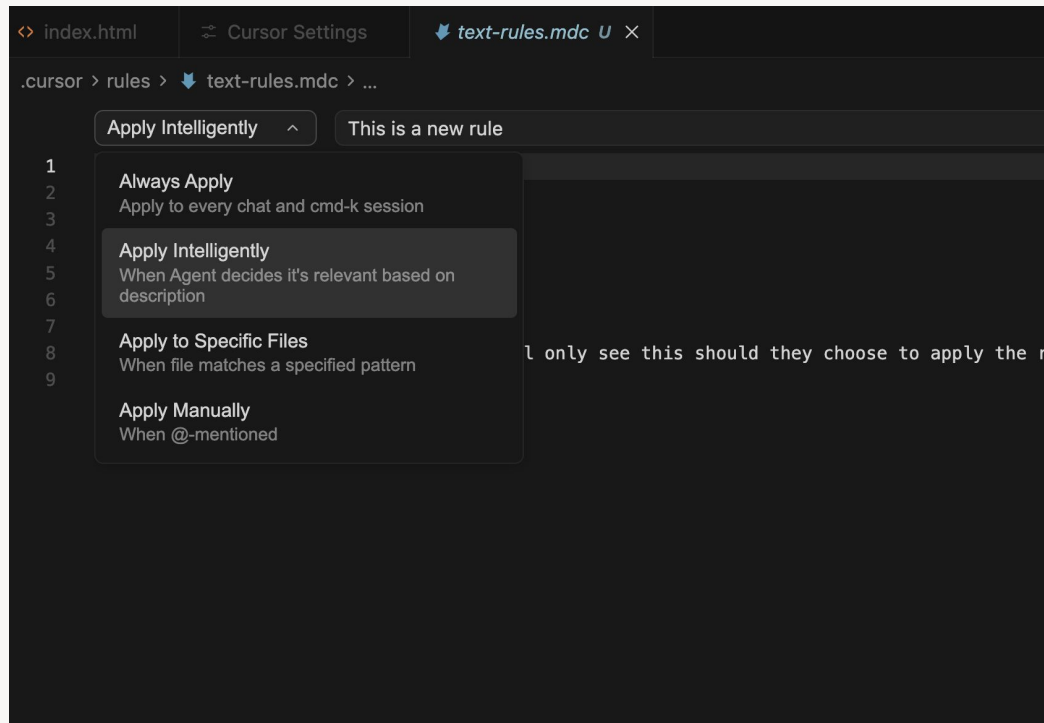
- Cursor has a bunch of default documentations built-in
- Type **@docs** + **Tab** to list and access any built-in documentations
- Adding documentation means Cursor can automatically use that docs as context without we having to go on Web and copy paste things
- Cursor also allows for adding our own docs
- Additionally, if there's no official documentation, Cursor accepts a Web URL and read through it
- Enable connection to external browser option to allow Cursor to perform internet search by itself

Cursor Rules



- Cursor Rules helps us customize how Cursor AI interacts with our code by guiding model's behaviour
- Access it from Cursor > Settings > Rules & Commands and add instructions
- Editor always listens to cursor rules file — it will always be passed in as context
 - **User Rules:** define and manage user level rules and preferences
 - **Project Rules:** define project specific instructions that Cursor will listen to while only working on this project
- Rules are useful when you have a particular way of doing things or if we want AI to follow specific instructions

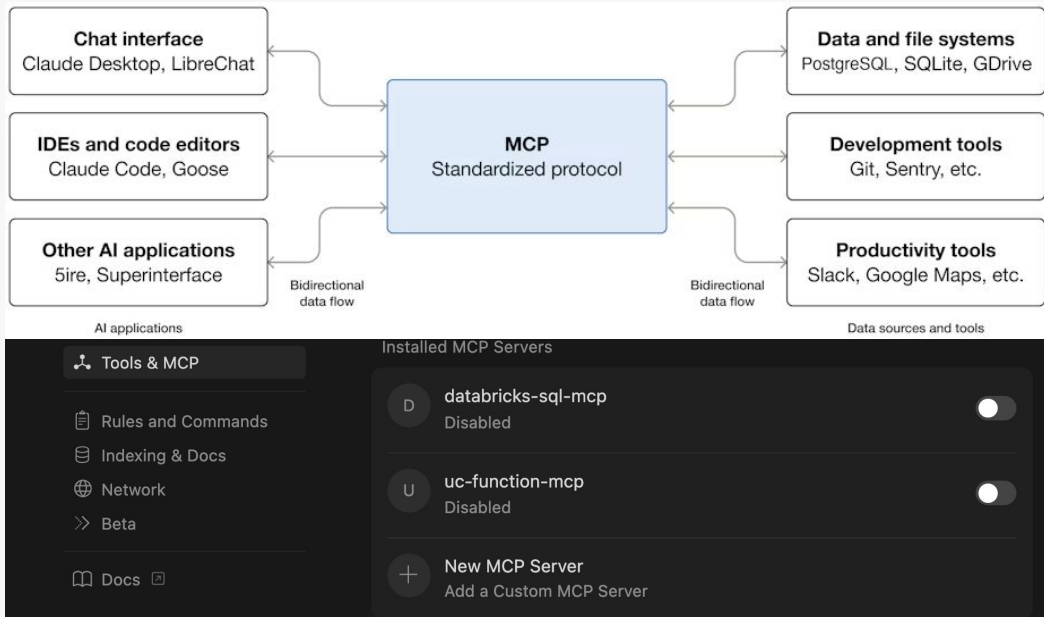
Cursor Rules



- Project rule supports 4 rule types determining how it passes context to Cursor AI
 - **Always apply:** apply the rule defined for every chat and terminal assistance session
 - **Apply intelligently:** rule applied when agent decides it's relevant based on the description of the rule
 - **Apply to specific files:** rule is used when file matches a specific pattern
 - **Apply manually:** rule is applied when explicitly mentioned with @
- You can create one or more rule files

Model Context Protocol

Model Context Protocol (MCP)



- MCP is standardized way of connecting any model to any tool (USB-C for AI applications)
- Connect AI assistants to the systems where data lives, including content repositories, business tools, and development environments
- MCP servers provides context to LLM to help produce better, more relevant responses
- Connect to MCP servers in Cursor going to Cursor Settings > Tools & MCP > New MCP Server